

AnyCloud平台适配蓝牙与WIFI一体无线模块ssv6158-M

1. 前言
2. 解压驱动：
 3. 修改驱动代码
 4. 修改Makefile文件
 5. 修改配置文件
 6. 编写编译脚本
 7. 驱动加载
8. 无线WiFi
 - 8.1 查看wlan0节点
 - 8.2 配网
9. BLE蓝牙
 - 9.1 查看蓝牙hci0节点
 - 9.2 查看节点
 - 9.3 蓝牙连接
10. 结束语

1. 前言

ssv6158-M是南硅出的一款蓝牙与WIFI一体的SDIO接口无线模块，除了要正常对接wifi功能外，还需要对接其蓝牙功能，本手册基于硬件一切正常的情况下，在Anycloud39AV100平台上进行适配，其他平台供参考。

2. 解压驱动：

```
unrar x L.SWB.23Q2.0000.00_r2489.rar
```

3. 修改驱动代码

南硅释放的驱动只支持SDIO4线模式，为了提供兼容性，修改为自适应SDIO1线模式和4线模式。

hwif/sdio/sdio.c文件：

```

diff --git a/hwif/sdio/sdio.c b/hwif/sdio/sdio.c
index a89611b..f03ab72 100755
--- a/hwif/sdio/sdio.c
+++ b/hwif/sdio/sdio.c
@@ -1924,15 +1924,16 @@ static void ssv6xxx_do_sdio_reset_reinit(struct
ssv6xxx_platform_data *pwlana_dat

    // Set host bus width
    sdio_claim_host(func);
-   host = func->card->host;
-   host->ios.bus_width = MMC_BUS_WIDTH_4;
-   host->ops->set_ios(host, &host->ios);
-   mdelay(20);
+   // host = func->card->host;
+   // host->ios.bus_width = MMC_BUS_WIDTH_4;
+   // host->ops->set_ios(host, &host->ios);
+   // mdelay(20);
    sdio_release_host(func);
+   host = func->card->host;

    // Set card bus width
    sdio_claim_host(func);
-   sdio_f0_writeb(func, SDIO_BUS_WIDTH_4BIT, SDIO_CCCR_IF, &err_ret);
+   sdio_f0_writeb(func, host->ios.bus_width, SDIO_CCCR_IF, &err_ret);
    sdio_release_host(func);
    CHECK_IO_RET(glue, err_ret);

```

4. 修改Makefile文件

调整Makefile文件，适配Anyccloud平台：

```

diff --git a/Makefile b/Makefile
index 553d856..05ff15e 100755
--- a/Makefile
+++ b/Makefile
@@ -118,9 +118,16 @@ strip:
     $(CROSS_COMPILE)strip $(KMODULE_NAME).ko --strip-unnneeded

install:
-    $(MAKE) INSTALL_MOD_DIR=$(INSTALL_PATH) -C $(KERNEL_OBJ_PATH) \
-    M=$(SSV_DRV_PATH) modules_install
-    depmod -a
+    # $(shell if [ ! -d $(INSTALL_PATH) ]; then mkdir $(INSTALL_PATH) ; fi;)
+    $(MAKE) INSTALL_MOD_DIR=$(INSTALL_PATH) -C $(KERNEL_OBJ_PATH) \
+    # M=$(SSV_DRV_PATH) modules_install
+    #depmod -a
+    @echo "install modules"
+    @rm -rf $(INSTALL_PATH)
+    @mkdir -p $(INSTALL_PATH)
+    @find $(SSV_DRV_PATH) -name *.ko | xargs -i cp {} $(INSTALL_PATH)
+    @find $(SSV_DRV_PATH) -name ssv6x5x-wifi.cfg | xargs -i cp {}
$(INSTALL_PATH)
+    @echo "install *.ko to $(INSTALL_PATH) successfully"

clean:
    $(MAKE) -C $(KERNEL_OBJ_PATH) M=$(SSV_DRV_PATH) ARCH=$(ARCH)
CROSS_COMPILE=$(CROSS_COMPILE) clean

```

5. 修改配置文件

修改config_common.mak配置文件：

```

# support BLE
ccflags-y += -DCONFIG_BLE

# BLE HCI BUS setting
# 1: over SDIO SSV_BLE_HCI_OVER_SDIO
# 0: over UART SSV_BLE_HCI_OVER_UART
ccflags-y += -DCONFIG_BLE_HCI_BUS=1

#ccflags-y += -DCONFIG_ENABLE_HOST_THERMAL

# for anyka portting
ccflags-y += -DCONFIG_PLATFORM_SDIO_BLOCK_SIZE=512

```

使能BLE支持

BLE使用SDIO接口

适配Anycloud平台需要添加CONFIG_PLATFORM_SDIO_BLOCK_SIZE宏定义

6. 编写编译脚本

指定内核路径和交叉编译器：

```
#!/bin/bash
CUR_DIR=$(dirname $(readlink -f "$0"))
echo "CUR_DIR=${CUR_DIR}"
SSV_DRV_PATH=${CUR_DIR}

KERNEL_OBJ_PATH=/home/xiaozijie/project/Anyccloud39AV100/V1.08/os/kbd
PLATFORM_TOOLCHAIN=/opt/arm-anyccloud-linux-uclibcgnueabi/bin/arm-anyccloud-linux-uclibcgnueabi-
INSTALL_PATH=/home/xiaozijie/workplace/wifi/sv6158/install

PLATFORM_ARCH=arm

# CPU
get_num_of_cpu_cores() {
    cpu_string=`cat /proc/cpuinfo | grep "cpu cores" | uniq`
    num_of_cpu_cores=${cpu_string:0-1:1}
}

get_num_of_cpu_cores
echo "##### num_of_cpu_cores:${num_of_cpu_cores} #####"

echo "##### step 1: make clean #####"
make -j${num_of_cpu_cores} -C $SSV_DRV_PATH KERNEL_OBJ_PATH=$KERNEL_OBJ_PATH
SSV_DRV_PATH=$SSV_DRV_PATH ARCH=$PLATFORM_ARCH CROSS_COMPILE=$PLATFORM_TOOLCHAIN
clean

echo "##### step 2: make modules #####"
make -j${num_of_cpu_cores} -C $SSV_DRV_PATH KERNEL_OBJ_PATH=$KERNEL_OBJ_PATH
SSV_DRV_PATH=$SSV_DRV_PATH ARCH=$PLATFORM_ARCH CROSS_COMPILE=$PLATFORM_TOOLCHAIN
modules

echo "##### step 3: make strip #####"
make -j${num_of_cpu_cores} -C $SSV_DRV_PATH SSV_DRV_PATH=$SSV_DRV_PATH
CROSS_COMPILE=$PLATFORM_TOOLCHAIN strip

echo "##### step 4: make install #####"
make -j${num_of_cpu_cores} -C $SSV_DRV_PATH KERNEL_OBJ_PATH=$KERNEL_OBJ_PATH
SSV_DRV_PATH=$SSV_DRV_PATH ARCH=$PLATFORM_ARCH CROSS_COMPILE=$PLATFORM_TOOLCHAIN
INSTALL_PATH=$INSTALL_PATH install
```

执行编译脚本，即可在安装目录找到编译好的ko及ssv6158需要的配置文件。

7. 驱动加载

因需要支持蓝牙，在内核代码中make menuconfig，打开以下选项：

```
[*] Networking support --->
<M> Bluetooth subsystem support --->
```

编译内核后会得到crc16.ko、bluetooth.ko两个ko。

驱动齐全后，进行驱动加载：

```
modprobe cfg80211
modprobe mac80211
modprobe mmc_core.ko
insmod /usr/modules/ak_mci.ko sdio_clk_mode=1

insmod /lib/modules/4.4.282/kernel/lib/crc16.ko
insmod /lib/modules/4.4.282/kernel/net/bluetooth/bluetooth.ko

insmod /usr/modules/ssv6x5x.ko stacfgpath=/etc/config/ssv6x5x-wifi.cfg
```

注意，加载ak_mci.ko驱动时，必须要添加sdio_clk_mode参数。

8. 无线WiFi

8.1 查看wlan0节点

```
[root@anyka ]$ ifconfig -a
wlan0      Link encap:Ethernet  HWaddr E0:CB:56:A9:28:9E
           UP BROADCAST MULTICAST  MTU:1500  Metric:1
           RX packets:0 errors:0 dropped:0 overruns:0 frame:0
           TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
           collisions:0 txqueuelen:1000
           RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

8.2 配网

```
wpa_supplicant -Dnl80211 -iwlan0 -c /etc/config/wpa_supplicant.conf -B
route del default
udhcpc -t 30 -T 1 -i wlan0 -b
```

注意将/etc/config/wpa_supplicant.conf文件的ssid和密码修改为无线路由器的。

9. BLE蓝牙

9.1 查看蓝牙hci0节点

将bluez工具包放置于Flash文件系统或TF卡中，设定环境变量让系统识别到hciconfig等工具。

环境变量设置参考脚本，脚本名假设：export.sh;

```
#!/bin/sh

export LD_LIBRARY_PATH="/usr/local/lib:$LD_LIBRARY_PATH"
export PATH="/usr/local/bin:/usr/local/libexec/bluetooth:$PATH"
```

注意环境变量脚本路径，使用点命令让其生效;

```
. ./export.sh
```

9.2 查看节点

```
[root@anyka]$ hciconfig -a
hci0:   Type: Primary  Bus: SDIO
        BD Address: E0:CB:56:A9:28:9F  ACL MTU: 251:8  SCO MTU: 0:0
        DOWN
        RX bytes:0 acl:0 sco:0 events:42 errors:0
        TX bytes:0 acl:0 sco:0 commands:42 errors:0
        Features: 0x00 0x00 0x00 0x00 0x60 0x00 0x00 0x00
        Packet type: DM1 DH1 HV1
        Link policy:
        Link mode: PERIPHERAL ACCEPT
```

9.3 蓝牙连接

为了能使bluez工具正常使用，需要将部分配置文件拷贝到对应目录。

- 1.将附件bluez目录的etc/dbus-1目录拷贝到设备的/etc目录和设备的/usr/local/etc目录，由于anyka平台/etc和/usr/local目录是不可写的，需要在制作rootfs时加入；
- 2.将附件bluez目录的usr/local/share/dbus-1拷贝到/usr/local/share目录，同样在nor flash要在制作rootfs时加入，nand flash可以读写则直接拷贝；
- 3.新增dbus用户，用于dbus运行，默认是messagebus，编译时设定为dbus了。

```
echo dbus:x:500:500::/home/dbus:/bin/sh >> /etc/passwd
```

- 4.系统启动时，在/var目录创建run/dbus目录

```
mkdir -p /var/run/dbus
```

- 5.运行dbus和bluetoothd服务

运行前必须要执行“hciconfig hci0 up”，让蓝牙节点起来。

```
hciconfig hci0 up  
dbus-daemon --system --print-pid --print-address  
bluetoothd -n -d &
```

- 6.执行bluetoothctl

```
bluetoothctl
```

- 7.在bluetoothctl窗口中依次输入如下命令，进行蓝牙广播

```
menu adv
name on
back
advertise on
menu gatt
register-service 11223344-1111-2222-3333-112233445566
#register-serviceyes
yes
register-characteristic 0x1234 read,write
#register-characteristic1
1
register-application
```

8.执行以上命令后，在安卓手机中使用“BLE调试宝”app，苹果手机使用“蓝牙调试助手”app，可以搜索到名为Bluez 5.60的蓝牙设备。

连接后在蓝牙服务界面，选择11223344-1111-2222-3333-112233445566服务，点击发送箭头，即可进行字符或16进制发送。

发送后，设备端可以收到对应的字符则蓝牙通信正常。

10. 结束语

由于默认工具及库文件没有strip，相关容量偏大，调试时应注意文件大小问题。